

Real-Time Cloth Simulation and Obstacle-Aware Control for Deformable Object Manipulation

Mehmet Taha Bekar

May 4, 2026

Abstract

This report develops a ROS2-compatible simulation and control framework for collaborative cloth manipulation. The work replaces a simplified position-based dynamics fabric model with an Isaac Sim and NVIDIA Newton Style3D simulation pipeline, integrates configurable anisotropic cloth parameters, and connects the simulator to an obstacle-aware Control Barrier Function controller. The system is evaluated entirely in simulation and prepared for future real-to-sim calibration using depth-based perception of real cloth state.

Contents

1	Introduction	5
1.1	Motivation	5
1.2	Proposed Direction	5
1.3	Problem Statement	6
1.4	Contributions	6
1.5	Report Organization	6
2	Background and Related Work	7
2.1	Technical Scope	7
2.2	Modeling Deformable Objects	7
2.3	Fabric Simulation and Newton Style3D	8
2.4	The Predecessor PBD–CBF Framework	9
3	Simulation Framework	11
3.1	Architecture	11
3.2	Isaac Sim and Newton Backend	11
3.3	Style3D Cloth Model	12
3.4	Scene Construction and Corner Pinning	13
3.5	ROS2 and Safety Interfaces	13
4	Control Barrier Function Controller	16
4.1	Controller Inputs and Outputs	16
4.2	Nominal Follower Command	16
4.3	Collision-Avoidance Barrier	17
4.4	Geometric Obstacle Jacobian	18
4.5	Stretch-Avoidance Barriers	18
4.6	QP Formulation and Implementation	19
4.7	Current Limitations	19

5	Experiments and Results	20
5.1	Experiment Setup	20
5.2	Experiment 1: CBF On vs CBF Off	20
5.3	Experiment 2: Cloth-Resolution Sweep	21
5.4	Experiment 3: Leader-Speed Sweep	21
5.5	Experiment 4: Contact-Friction Sweep	23
5.6	Experiment 5: Material-Stiffness Sweep	23
6	Discussion, Conclusion, and Future Work	27
6.1	Discussion	27
6.2	Conclusion	28
6.3	Future Work	28

List of Figures

3.1	High-level ROS2 data flow for the Isaac Sim, Newton, Style3D, and CBF simulation framework.	12
3.2	Isaac Sim scene used for Newton Style3D cloth simulation.	15
4.1	Leader input velocity flow used by the nominal follower-command generator.	17
5.1	Experiment 1 minimum obstacle distance. CBF off leads to table penetration; CBF on avoids negative signed distance.	22
5.2	Experiment 1 corner trajectories. With CBF enabled, the follower corners move upward near the obstacle.	23
5.3	Experiment 1 CBF-off. The sheet is dragged into the table obstacle.	24
5.4	Experiment 1 CBF-on. The follower edge lifts as the table becomes active in the barrier constraint.	24
5.5	Experiment 2 screenshots for increasing cloth resolution: 20x20, 56x56, 72x72, 128x128.	25
5.6	Experiment 3 minimum obstacle distance for three leader speeds. The 1.00 m/s run reaches only 1.15 mm clearance and is treated as touching/failure rather than successful margin enforcement.	25
5.7	Experiment 4 contact-friction sweep. From left to right: $\mu = 0.05, 0.50, 1.00$, and 4.00.	25
5.8	Experiment 5 stiffness sweep. From left to right: low, mid, and high stiffness presets.	26

List of Tables

2.1	Practical comparison of common deformable-object models for real-time robotic manipulation.	8
2.2	Design implications inherited from the predecessor framework.	10
3.1	Main modules in the simulation framework.	12
3.2	Current active simulation parameters.	13
3.3	Current Style3D material and contact parameters.	14
3.4	Main ROS2 topics used by the simulation framework.	15
4.1	Main ROS2 interfaces of the CBF controller.	19
5.1	Primary metrics used in the experiment summaries.	21
5.2	Experiment 1 CBF on/off results. The configured safety margin is 0.05 m. .	21
5.3	Experiment 2 resolution-sweep results. The cloth physical size is held fixed while particle count increases.	22

Chapter 1

Introduction

1.1 Motivation

Robotic manipulation of deformable objects is still less mature than rigid-object manipulation. Tasks such as cloth transportation, bed-sheet handling, garment folding, and composite layup require robots to control objects whose shapes change continuously under gravity, contact, friction, and external forces [1, 2]. The state is high-dimensional, only partially observable, and difficult to predict in real time.

This report focuses on sheet-like deformable objects, using cloth as the main simulated case study. Woven fabrics are difficult to simulate because they can resist deformation differently along warp, weft, and shear directions. Their motion is also affected by bending, sagging, wrinkles, folds, frictional sliding, and obstacle contact. For robotic manipulation, the simulator must be fast enough for closed-loop use and realistic enough that controller decisions based on the simulated sheet are not misleading.

The larger target is collaborative manipulation. A human or leader robot moves part of the sheet, while assistant robots support other parts and avoid unsafe interactions with the environment. Previous work showed that real-time deformable-object simulation can be embedded inside a Control Barrier Function (CBF) controller for safe collaborative manipulation [3, 4]. This project keeps that control idea but replaces the simplified two-dimensional fabric simulation backend with an Isaac Sim and NVIDIA Newton Style3D pipeline.

1.2 Proposed Direction

NVIDIA Newton provides a GPU-oriented physics framework built on Warp and OpenUSD, and its Style3D cloth solver exposes anisotropic material parameters such as weft, warp, shear, and bending stiffness [5, 6]. These parameters are more suitable for woven fabric than a generic isotropic distance-constraint sheet.

The project ports the simulation and control stack toward ROS2, Isaac Sim, and Newton Style3D. The simulation represents the sheet as a gridded cloth model, pins corner regions to

controlled gripper targets, and evaluates fabric behavior under gravity, contact, and obstacle interaction. The control side computes nominal motion for follower corners and modifies it when necessary through a CBF-style safety layer. The current implementation uses a lightweight geometric approximation of the obstacle Jacobian; the software is kept modular so future finite-difference Jacobians from auxiliary Newton rollouts can replace it.

The same framework is also prepared for future real-to-sim calibration. A depth camera and visual markers would estimate the real cloth state, compare it with the simulated sheet state, and tune simulation parameters online or through repeated parameter sweeps. That loop is not closed in this report.

1.3 Problem Statement

The problem addressed in this report is the development and evaluation of a real-time, ROS2-compatible simulation and obstacle-aware control framework for collaborative manipulation of sheet-like deformable objects, with fabric sheets as the main target.

The framework should satisfy four requirements. It should simulate cloth-like behavior with anisotropic parameters, run fast enough for interactive or closed-loop use, expose a clean ROS2 interface for commanding cloth corner points, and support CBF-style obstacle avoidance while leaving a path toward future rollout-based Jacobian estimation and real-to-sim parameter optimization.

1.4 Contributions

The contributions are: a standalone Isaac Sim/Newton Style3D cloth-simulation pipeline; a patch-based corner-pinning method for more stable gripper interaction; a ROS2-oriented CBF controller for follower-corner obstacle avoidance; and simulation experiments covering CBF behavior, cloth resolution, leader speed, contact friction, and material stiffness.

1.5 Report Organization

Chapter 2 reviews deformable-object simulation, Newton Style3D, CBF safety control, and the predecessor PBD-CBF framework. Chapter 3 presents the Isaac Sim and Newton simulation architecture. Chapter 4 describes the ROS2 CBF controller. Chapter 5 presents the experiments and results. Chapter 6 discusses limitations, concludes the report, and outlines future work, including the planned perception and real-to-sim calibration extension.

Chapter 2

Background and Related Work

2.1 Technical Scope

This chapter summarizes the background needed for the implemented system: real-time deformable-object models, fabric simulation, Newton Style3D, and the predecessor PBD-CBF framework. The emphasis is practical. The simulator must be realistic enough to support controller decisions, but fast enough to run in a manipulation loop.

2.2 Modeling Deformable Objects

A deformable object cannot be represented by a single rigid-body pose. Its state is distributed over particles, mesh vertices, material points, or finite elements. In robotic manipulation, this creates two difficulties: the state is high-dimensional, and cameras or markers usually observe only part of it. Simulation is therefore often used as an internal prediction model.

The main modeling families relevant to this project are mass-spring systems, finite-element methods, PBD/XPBD, and projective-dynamics cloth solvers. Their practical differences are summarized in Table 2.1.

Mass-spring systems model the object as point masses connected by elastic springs. They are inexpensive per step, but realistic cloth usually requires high stretch stiffness. With explicit integration, high stiffness forces very small time steps and can cause instability. Implicit mass-spring formulations improve stability but require solving linear systems [7, 8].

Finite-element methods discretize continuum mechanics over elements and derive forces from material strain energy. This gives FEM a stronger physical basis than simple spring networks and makes it useful when stress, strain, and measurable material parameters are important [9]. For this project, however, high-fidelity FEM is treated as a possible offline reference or calibration tool, not as the online simulation backend.

PBD represents deformable objects as particles constrained by geometric relationships. Instead of integrating elastic forces directly, it predicts particle positions and projects them

Table 2.1: Practical comparison of common deformable-object models for real-time robotic manipulation.

Model	Strength	Limitation
Mass-spring systems	Simple particle-and-spring representation; easy to implement.	Explicit versions become unstable for stiff cloth; spring constants are hard to map to real fabric properties.
FEM	Strong physical basis with measurable material parameters.	Usually too expensive for tight real-time control with contact-rich cloth.
PBD/XPBD	Stable, fast, and well suited to interactive manipulation and auxiliary rollouts.	Classical PBD has iteration-dependent stiffness; simplified constraints are weak for calibrated 2D fabric.
Projective dynamics / Style3D	Implicit-style stability with efficient local/global iterations and anisotropic cloth parameters.	Still approximate; contact and calibration require careful tuning.

to satisfy constraints. This makes PBD stable and computationally attractive for real-time robotics. XPBD improves classical PBD by adding compliance, reducing the dependence of effective stiffness on iteration count and time step [10, 11]. These properties explain why the predecessor framework used PBD/XPBD.

For thin sheet-like deformable objects such as fabric sheets, a simple PBD sheet is not enough for the goals of this report. Real woven cloth behaves differently along warp, weft, and shear directions, and bending strongly affects wrinkles, sagging, and contact. This motivates the move toward Newton Style3D.

2.3 Fabric Simulation and Newton Style3D

Cloth sheets are difficult to simulate because they can be nearly inextensible in-plane while remaining flexible in bending and shear. Baraff and Witkin showed that implicit integration can take larger steps than explicit methods for stiff cloth systems [7]. Later work reformulated implicit mass-spring simulation as an optimization problem with efficient local/global iterations [8], and Projective Dynamics generalized this idea for fast simulation based on alternating local projections and global solves [12].

NVIDIA Newton Style3D follows this implicit-style, projective-dynamics direction. In Newton, Style3D can build cloth from grids or triangle meshes and expose anisotropic fabric parameters [5, 6]. Newton is also relevant because it is GPU-oriented and compatible with robotics simulation workflows. Isaac Sim 6.0 can run Newton through a physics backend, allowing the cloth simulation to be placed inside a scene with USD assets, obstacles, cameras, and ROS2 integration [13].

2.4 The Predecessor PBD–CBF Framework

The direct predecessor of this work is the ROS1 framework by Aksoy and Wen for collaborative manipulation of deformable objects [3, 4], including the composite-layup package used as a software reference [14]. The predecessor discretizes deformable objects into particles, simulates their motion with PBD-style constraints, and uses the simulated object state inside a CBF-based quadratic program.

The manipulation structure is leader–assistant control. A human or leader robot moves one part of the object. Assistant robots hold other points and follow a desired formation. The nominal controller moves each assistant toward its target position. The CBF-QP modifies that command only when safety constraints become active. The important idea is that safety is computed from the simulated deformable object, not only from the robot gripper positions.

For collision avoidance, the predecessor uses

$$h^{\text{coll}}(x) = f(x) - d^{\text{offset}}, \quad (2.1)$$

where $f(x)$ is the minimum distance between the deformable object and the environment, and d^{offset} is the safety margin. The CBF condition requires

$$\dot{h}^{\text{coll}}(x, u) \geq -\alpha(h^{\text{coll}}(x)), \quad (2.2)$$

where $\alpha(\cdot)$ is a class- $\mathcal{K}$ function. Since $f(x)$ depends on the full deformable-object state while the robot controls only a few grasped points, Aksoy and Wen approximate

$$\nabla f(x)g(x, u) \approx J^{\text{coll}}(x)u, \quad (2.3)$$

which gives the linear CBF constraint

$$J^{\text{coll}}(x)u \geq -\alpha(h^{\text{coll}}(x)). \quad (2.4)$$

The predecessor estimated this Jacobian with auxiliary PBD simulations. The resulting QP is

$$\begin{aligned} \min_u \quad & \frac{1}{2} \|u - u_{\text{nom}}\|^2 \\ \text{s.t.} \quad & J^{\text{coll}}(x)u \geq -\alpha(h^{\text{coll}}(x)), \\ & \text{additional stretch, proximity, and velocity constraints.} \end{aligned} \quad (2.5)$$

This project preserves the leader–assistant structure, simulator-based distance estimate, and QP-CBF safety filter. It replaces the ROS1 PBD fabric backend because the predecessor’s sheet model was difficult to calibrate, did not expose direct warp/weft/shear material parameters, was less attractive for high-resolution contact-rich cloth, and did not fit the ROS2/Isaac Sim deployment target. Table 2.2 summarizes the design consequences inherited from the predecessor.

Table 2.2: Design implications inherited from the predecessor framework.

Predecessor element	Decision here	Reason
CBF-QP safety filter	Preserve	Minimally changes nominal commands only when constraints become active.
Simulator-in-the-loop control	Preserve	Safety depends on the full deformable-object shape, not only on gripper positions.
Auxiliary rollout Jacobian	Keep as future option	More accurate than the current geometric Jacobian, but more expensive.
ROS1 PBD fabric backend	Replace	Limited anisotropy, calibration difficulty, and weaker Isaac Sim/ROS2 integration.
Simplified material parameters	Replace with Style3D parameters	Style3D exposes warp, weft, shear, bending, damping, and contact-related parameters.

Chapter 3

Simulation Framework

3.1 Architecture

The simulation framework replaces the predecessor ROS1/PBD cloth backend with a ROS2-compatible Isaac Sim and NVIDIA Newton pipeline. Isaac Sim provides the robotics scene, visualization, and ROS2 integration. Newton provides the GPU-oriented physics backend. Style3D provides the cloth solver and anisotropic fabric parameters. Additional ROS2 nodes handle teleoperation, obstacle-distance computation, CBF command filtering, and logging. The main software modules are listed in Table 3.1.

The implementation uses a leader–follower convention. The two upper cloth corners are the leader side, corresponding to the human-held or teleoperated side. The two lower corners are the follower side, corresponding to the robot-controlled side.

Figure 3.1 summarizes the data flow. Keyboard teleoperation publishes leader-corner velocity commands. Isaac Sim applies these commands, advances the Newton/Style3D cloth, and publishes the deformable-object state. The obstacle-distance node computes the closest cloth–obstacle distance. The CBF node receives the leader commands, corner poses, and obstacle-distance message, then publishes safe follower-corner velocity commands back to Isaac Sim.

3.2 Isaac Sim and Newton Backend

Isaac Sim is used because the project needs a robotics-oriented scene rather than a standalone cloth solver. It provides USD scene structure, rendering, ROS2 bridge support, and a path for adding robot arms and calibration targets. Newton is used because it provides a GPU-oriented physics backend built around Warp and includes deformable-object solvers relevant to cloth manipulation [5, 13]. Style3D is selected because it directly supports cloth grids and anisotropic fabric stiffness parameters [6].

The simulation runs as a standalone Isaac Sim Python application. The script creates the USD stage, visual meshes, Newton model, Style3D cloth, and ROS2 interfaces. The

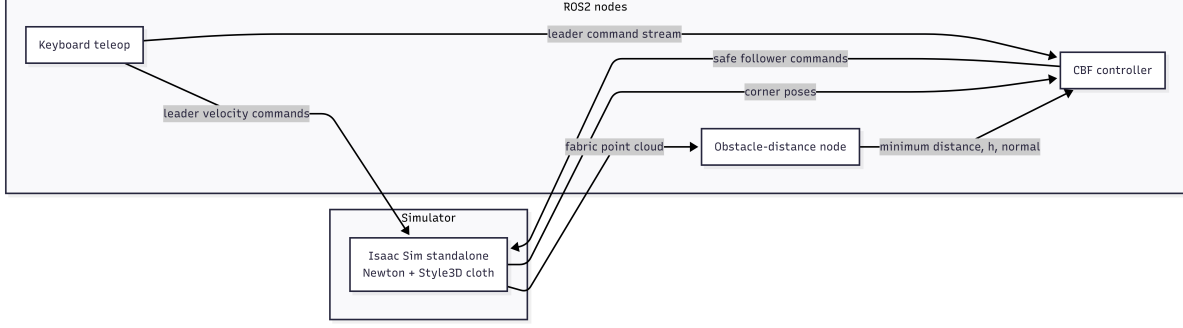


Figure 3.1: High-level ROS2 data flow for the Isaac Sim, Newton, Style3D, and CBF simulation framework.

Table 3.1: Main modules in the simulation framework.

Module	Role
<code>fabric_newton_isaacsim_standalone.py</code>	Starts Isaac Sim, builds the USD scene, creates the Newton Style3D cloth, applies corner pinning, advances the simulation, and publishes ROS2 state.
<code>fabric_sim_bringup</code>	Starts the full stack from one YAML configuration file.
<code>fabric_teleop</code>	Publishes leader-left and leader-right velocity commands from keyboard input.
<code>fabric_obstacle_distance</code>	Reads the simulated cloth point cloud and computes signed distance to the table obstacle.
<code>fabric_cbf_controller</code>	Filters nominal follower commands using obstacle and stretch CBF constraints.

Newton solver advances inside the Isaac Sim update loop, while ROS2 callbacks are spun during the same loop so command input and state publishing remain synchronized.

3.3 Style3D Cloth Model

The cloth is created with Newton’s Style3D grid helper. The default configuration uses a 56×56 cell grid, giving $57 \times 57 = 3249$ particles. The cell size is 0.015 m, so the cloth sheet is approximately 0.84×0.84 m.

Style3D exposes triangle stiffness in the weft, warp, and shear directions, area stiffness, edge bending stiffnesses, damping, and contact parameters. These are not yet calibrated real material constants. The active grid, solver, and scene parameters are listed in Table 3.2; the material and contact parameters are listed in Table 3.3.

Table 3.2: Current active simulation parameters.

Group	Parameter	Value	Meaning
Grid	<code>grid_x, grid_y</code>	56, 56	Cloth cells in each direction.
Grid	<code>cell_x, cell_y</code>	0.015, 0.015 m	Cloth cell spacing.
Initial state	<code>cloth_start_z</code>	0.72 m	Initial cloth height.
Particles	<code>mass_per_particle</code>	0.003 kg	Mass assigned to each particle.
Particles	<code>particle_radius</code>	0.008 m	Radius used for contact and distance correction.
Obstacle	<code>table_hx, table_hy, table_hz</code>	0.22, 0.22, 0.16 m	Half-extents of the table box.
Solver	<code>substeps</code>	8	Newton substeps per rendered frame.
Solver	<code>iterations</code>	10	Style3D nonlinear iterations.
Solver	<code>linear_iterations</code>	16	Linear iterations inside the Style3D solve.
Pinning	<code>gripper_size</code>	3	3×3 particle patch at each corner.
Publishing	<code>state_publish_every</code>	3 frames	Full cloth point-cloud publish period.

3.4 Scene Construction and Corner Pinning

The Isaac Sim scene is created programmatically. The script creates a USD stage, sets meter units and z -up orientation, adds lighting, and builds a ground plane, table obstacle, cloth mesh, and visual corner markers. The Newton model mirrors the relevant static collision geometry. The rendered USD cloth mesh is updated from Newton particle positions after each simulation step. Figure 3.2 shows the resulting simulation window.

The cloth is manipulated through four kinematic corner targets. To avoid unrealistic deformation from single-particle constraints, each target controls a square patch of particles. With `gripper_size=3`, each corner pins a 3×3 patch, for 36 constrained particles total.

For each pinned particle, the simulator stores its initial offset from the corresponding corner target and overwrites its position as

$$\mathbf{x}_i(t) = \mathbf{x}_c^{\text{target}}(t) + \Delta \mathbf{x}_i. \quad (3.1)$$

This hard-pinning operation is applied before and after each Newton substep to prevent drift. It is a stable kinematic boundary condition, but not a physical gripper model. Grasp compliance, finger geometry, and slippage are outside the current implementation.

3.5 ROS2 and Safety Interfaces

The simulator publishes cloth state and corner poses, and subscribes to leader and follower velocity commands. Table 3.4 lists the main topics.

Table 3.3: Current Style3D material and contact parameters.

Group	Parameter	Value	Meaning
Triangle stretch	<code>tri_ke_weft</code> , <code>tri_ke_warp</code> , <code>tri_ke_shear</code>	$(3.0 \times 10^4, 3.0 \times 10^4, 6.0 \times 10^3)$	Weft, warp, and shear stiffness.
Triangle area	<code>tri_ka</code>	5.0×10^3	Area-preservation stiffness.
Triangle damping	<code>tri_kd</code>	5.0×10^{-5}	Triangle-level damping.
Edge bending	<code>edge_ke_weft</code> , <code>edge_ke_warp</code> , <code>edge_ke_shear</code>	$(2.0 \times 10^{-3}, 1.0 \times 10^{-3}, 5.0 \times 10^{-4})$	Directional bending stiffness.
Edge damping	<code>edge_kd</code>	1.0×10^{-5}	Edge-level damping.
Soft contact	<code>contact_radius</code>	0.010 m	Contact query radius.
Soft contact	<code>contact_margin</code>	0.012 m	Contact activation margin.
Soft contact	<code>contact_ke</code>	1.0×10^3	Contact stiffness.
Soft contact	<code>contact_kd</code>	1.0×10^{-2}	Contact damping.
Soft contact	<code>contact_mu</code>	2.0	Contact friction coefficient.

The obstacle-distance node currently assumes a known axis-aligned table box. For each cloth particle $\mathbf{p}$, box center $\mathbf{c}$, and box half-extents $\mathbf{b}$, it computes

$$\mathbf{q} = |\mathbf{p} - \mathbf{c}| - \mathbf{b}, \quad (3.2)$$

$$d_{\text{box}}(\mathbf{p}) = \|\max(\mathbf{q}, 0)\|_2 + \min(\max(q_x, q_y, q_z), 0). \quad (3.3)$$

The node subtracts the particle radius and reports the minimum over all particles. The CBF barrier value is

$$h = d_{\min} - d_{\text{safety}}, \quad (3.4)$$

where $d_{\text{safety}} = 0.05$ m. This box-specific layer is sufficient for the table experiments, but a future system should replace it with a general nearest-distance or SDF query for arbitrary obstacles and robot bodies.

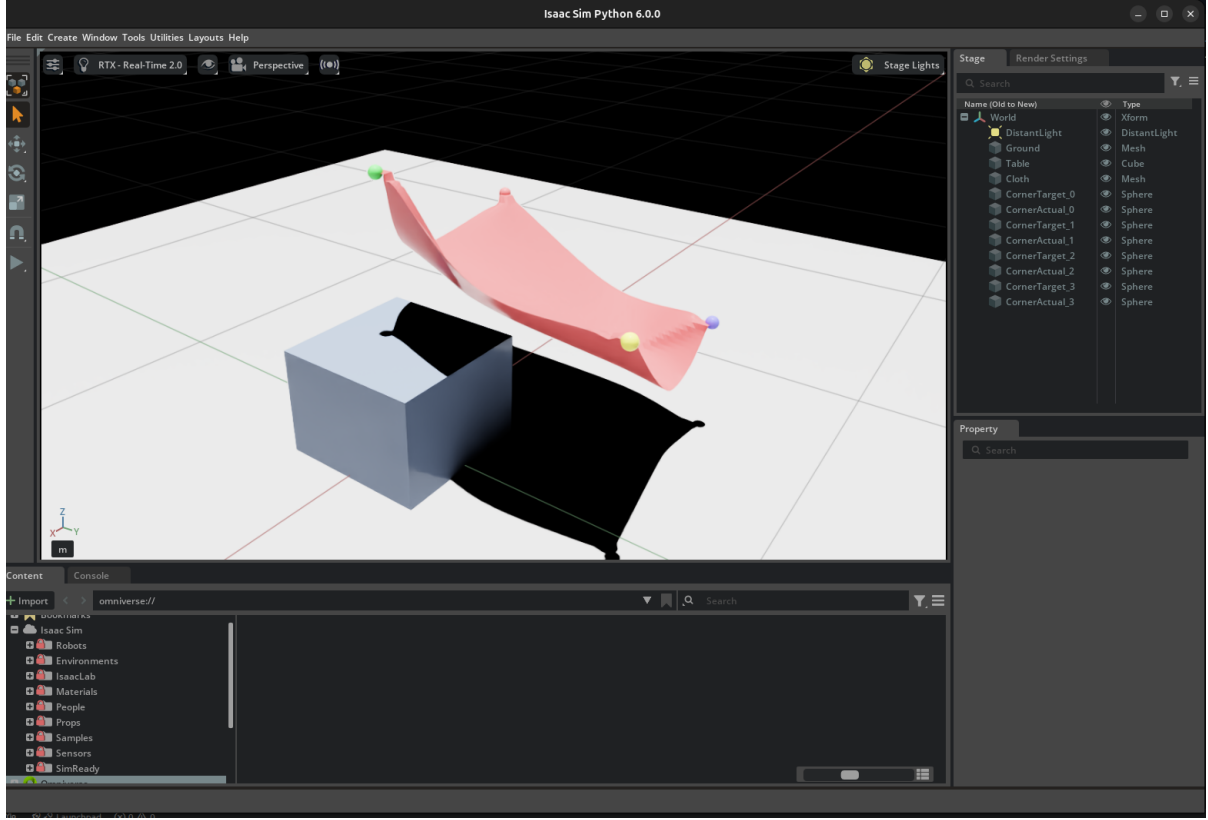


Figure 3.2: Isaac Sim scene used for Newton Style3D cloth simulation.

Table 3.4: Main ROS2 topics used by the simulation framework.

Topic	Type	Role
/fabric/leader_left/cmd_vel	TwistStamped	Leader-left velocity input from teleoperation.
/fabric/leader_right/cmd_vel	TwistStamped	Leader-right velocity input from teleoperation.
/fabric/leader_left/cbf_input_cmd_vel	TwistStamped	Republished leader-left input consumed by the CBF node.
/fabric/leader_right/cbf_input_cmd_vel	TwistStamped	Republished leader-right input consumed by the CBF node.
/fabric/follower_left/cbf_cmd_vel	TwistStamped	Safe follower-left velocity from the CBF node.
/fabric/follower_right/cbf_cmd_vel	TwistStamped	Safe follower-right velocity from the CBF node.
/fabric/sim/corners	PoseArray	Simulated poses of the four cloth corners.
/fabric/sim/state	PointCloud2	Full simulated particle cloud.
/fabric/obstacle_distance	custom	Minimum cloth-obstacle distance and closest-point data.
/fabric/cbf/debug	custom	CBF mode, barrier values, Jacobian, and command data.

Chapter 4

Control Barrier Function Controller

4.1 Controller Inputs and Outputs

The controller modifies follower-corner velocity commands when the simulated cloth approaches the table obstacle or violates geometric stretch limits. It receives leader input velocities, four corner poses, the cloth–obstacle distance message, and the obstacle normal. It publishes filtered follower commands back to Isaac Sim.

Let

$$\mathbf{u} = \begin{bmatrix} \mathbf{u}_{FL} \\ \mathbf{u}_{FR} \end{bmatrix} \in \mathbb{R}^6 \quad (4.1)$$

be the commanded velocity vector for the follower-left and follower-right corners. Figure 4.1 shows the leader-input flow used to generate the nominal follower command.

4.2 Nominal Follower Command

The nominal command is used when no safety constraint requires correction. Each follower first inherits the corresponding leader-side input velocity. A slow formation-recovery term then pulls the followers back toward their initial leader-to-follower offsets.

When the first valid corner-pose message is received, the controller stores

$$\mathbf{r}_{FL}^0 = \mathbf{p}_{FL}(0) - \mathbf{p}_{LL}(0), \quad (4.2)$$

$$\mathbf{r}_{FR}^0 = \mathbf{p}_{FR}(0) - \mathbf{p}_{LR}(0). \quad (4.3)$$

The desired follower positions are

$$\mathbf{p}_{FL}^d = \mathbf{p}_{LL} + \mathbf{r}_{FL}^0, \quad (4.4)$$

$$\mathbf{p}_{FR}^d = \mathbf{p}_{LR} + \mathbf{r}_{FR}^0. \quad (4.5)$$

After a deadband, the recovery velocity is

$$\mathbf{u}_{F,i}^{\text{rec}} = \text{sat}_{v_{\text{rec,max}}} \left(k_{\text{rec}} (\mathbf{p}_{F,i}^d - \mathbf{p}_{F,i}) \right), \quad i \in \{L, R\}. \quad (4.6)$$

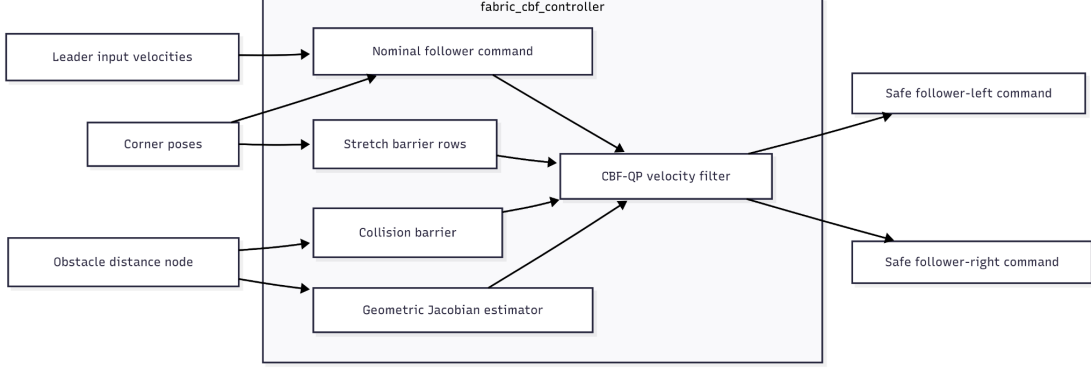


Figure 4.1: Leader input velocity flow used by the nominal follower-command generator.

The final nominal command is

$$\mathbf{u}_{\text{nom}} = \begin{bmatrix} \mathbf{v}_{LL}^{\text{input}} + \mathbf{u}_{FL}^{\text{rec}} \\ \mathbf{v}_{LR}^{\text{input}} + \mathbf{u}_{FR}^{\text{rec}} \end{bmatrix}. \quad (4.7)$$

The recovery term prevents the followers from remaining permanently displaced after the obstacle is no longer active.

4.3 Collision-Avoidance Barrier

The collision barrier follows the predecessor CBF formulation. Let $f(\mathbf{x})$ be the minimum distance between the deformable object state $\mathbf{x}$ and the obstacle. The barrier is

$$h_{\text{coll}}(\mathbf{x}) = f(\mathbf{x}) - d_{\text{offset}}, \quad (4.8)$$

with condition

$$\dot{h}_{\text{coll}}(\mathbf{x}, \mathbf{u}) \geq -\alpha_{\text{coll}}(h_{\text{coll}}(\mathbf{x})), \quad (4.9)$$

where $\alpha_{\text{coll}}(\cdot)$ is an extended class- $\mathcal{K}$ function [3, 15]. In the implemented system, the distance node publishes

$$h = d_{\text{min}} - d_{\text{safety}}, \quad (4.10)$$

with $d_{\text{safety}} = 0.05$ m.

For a deformable object, $\dot{h}_{\text{coll}}$ is difficult to compute analytically. The predecessor approximated

$$\nabla f(\mathbf{x})g(\mathbf{x}, \mathbf{u}) \approx J_{\text{coll}}(\mathbf{x})\mathbf{u} \quad (4.11)$$

using auxiliary PBD simulations [3]. The current controller keeps the same linear constraint form,

$$J_{\text{coll}}(\mathbf{x})\mathbf{u} \geq -\alpha_{\text{coll}}(h), \quad (4.12)$$

but uses a geometric Jacobian approximation. The default α_{coll} is piecewise linear:

$$\alpha_{\text{coll}}(h) = \begin{cases} k_{\text{near}}h, & h \leq h_{\text{switch}}, \\ k_{\text{near}}h_{\text{switch}} + k_{\text{far}}(h - h_{\text{switch}}), & h > h_{\text{switch}}. \end{cases} \quad (4.13)$$

The current configuration uses $k_{\text{near}} = 5.0$, $k_{\text{far}} = 1.0$, and $h_{\text{switch}} = 0.05$ m.

4.4 Geometric Obstacle Jacobian

The distance node reports the closest cloth particle, the closest point on the obstacle, and the outward normal

$$\mathbf{n} = \frac{\mathbf{p}_{\text{closest}} - \mathbf{p}_{\text{obs}}}{\|\mathbf{p}_{\text{closest}} - \mathbf{p}_{\text{obs}}\|}. \quad (4.14)$$

The controller assumes that follower-corner velocity changes move the closest cloth point partly along this normal. The Jacobian row is

$$J_{\text{coll}} = [w_L \mathbf{n}^T \quad w_R \mathbf{n}^T] \in \mathbb{R}^{1 \times 6}, \quad (4.15)$$

where w_L and w_R are influence weights for the follower-left and follower-right corners.

The implementation supports equal weighting, distance-based weighting from the closest particle to each follower corner, and left/right weighting from the closest particle's horizontal position. All modes normalize $w_L + w_R = 1$.

This Jacobian is a heuristic. It is fast, but it can be inaccurate near folds, contact switches, edge contacts, or cases where leader motion dominates the closest-point motion. The controller is written so this estimator can later be replaced by finite-difference Newton rollouts without changing the QP layer.

4.5 Stretch-Avoidance Barriers

The controller also includes geometric stretch constraints between selected held corners. For a pair (i, j) , the barrier is

$$h_{ij}^{\text{stretch}}(\mathbf{x}) = d_{ij}^{\text{max}} - \|\mathbf{p}_j - \mathbf{p}_i\|. \quad (4.16)$$

This follows the predecessor maximum-distance idea [3]. With

$$\mathbf{a}_{ij} = \frac{\mathbf{p}_j - \mathbf{p}_i}{\|\mathbf{p}_j - \mathbf{p}_i\|}, \quad (4.17)$$

and with point i directly commanded by $\dot{\mathbf{p}}_i = \mathbf{u}_i$, the derivative is

$$\dot{h}_{ij}^{\text{stretch}} = -\mathbf{a}_{ij}^T (\dot{\mathbf{p}}_j - \mathbf{u}_i) \geq -\alpha_{\text{stretch}} (h_{ij}^{\text{stretch}}). \quad (4.18)$$

Rearranging this inequality gives the CBF row for controllable follower point i :

$$\mathbf{a}_{ij}^T \mathbf{u}_i \geq -\alpha_{\text{stretch}} (h_{ij}^{\text{stretch}}) + \mathbf{a}_{ij}^T \dot{\mathbf{p}}_j. \quad (4.19)$$

The default active pairs are follower-left to leader-left, follower-right to leader-right, and follower-left to follower-right. These limits are geometric; they are not measured tensile limits of the material.

4.6 QP Formulation and Implementation

The controller computes the safe command closest to the nominal command:

$$\mathbf{u}^* = \arg \min_{\mathbf{u} \in \mathbb{R}^6} \frac{1}{2} \|\Gamma(\mathbf{u} - \mathbf{u}_{\text{nom}})\|_2^2 \quad (4.20)$$

$$\text{s.t. } J_{\text{coll}} \mathbf{u} \geq -\alpha_{\text{coll}}(h), \quad \text{if obstacle CBF is active,} \quad (4.21)$$

$$A_{\text{stretch}} \mathbf{u} \geq \mathbf{b}_{\text{stretch}}, \quad \text{for active stretch rows,} \quad (4.22)$$

$$-\mathbf{u}_{\text{max}} \leq \mathbf{u} \leq \mathbf{u}_{\text{max}}. \quad (4.23)$$

Here $\Gamma = \text{diag}(\gamma_x, \gamma_y, \gamma_z, \gamma_x, \gamma_y, \gamma_z)$, and the default maximum follower speed is 0.20 m/s.

The implementation avoids a heavy external QP dependency. For a single obstacle constraint, it uses analytic weighted projection onto one half-space with box clipping. With multiple active constraints, it uses iterative weighted half-plane projection with velocity bounds. If the nominal command already satisfies all active inequalities, the output remains nominal. Table 4.1 lists the controller’s main ROS2 interfaces.

Table 4.1: Main ROS2 interfaces of the CBF controller.

Topic	Type	Role
/fabric/leader_left/cbf_input_cmd_vel	TwistStamped	Leader-left velocity input to the controller.
/fabric/leader_right/cbf_input_cmd_vel	TwistStamped	Leader-right velocity input to the controller.
/fabric/sim/corners	PoseArray	Current follower and leader corner poses.
/fabric/obstacle_distance	ObstacleDistance	Minimum distance, barrier value, closest points, and normal.
/fabric/follower_left/cbf_cmd_vel	TwistStamped	Safe follower-left velocity command.
/fabric/follower_right/cbf_cmd_vel	TwistStamped	Safe follower-right velocity command.
/fabric/cbf/debug	CbfDebug	Mode, barrier values, Jacobian, nominal/safe commands, and timing.

4.7 Current Limitations

The controller is a practical safety filter, not a full safety guarantee for a physical cloth-manipulation system. The geometric Jacobian does not measure the true sensitivity of cloth–obstacle distance to follower motion. The minimum-distance function is nonsmooth when the closest point switches. If the leader drives the cloth too quickly, the follower velocity bound may be insufficient to maintain the 0.05 m margin. Stretch avoidance is based on corner distances rather than distributed strain. Finally, the controller commands idealized cloth corner targets, not full robot arms with kinematic limits, gripper compliance, or robot-body collision constraints.

Chapter 5

Experiments and Results

5.1 Experiment Setup

This chapter evaluates the Isaac Sim, Newton Style3D, ROS2, and CBF controller pipeline using the recorded experiment logs in the project result files. The goal is to verify that the simulation runs repeatable cloth-manipulation experiments, records useful metrics, and shows the expected obstacle-avoidance and material-parameter behavior.

All experiments use a square cloth sheet, a rectangular table obstacle, and four kinematic corner targets. The two leader corners are driven by scripted velocity commands. The two follower corners are either driven by nominal commands or by commands filtered through the external ROS2 CBF controller. The controller receives corner poses and cloth-obstacle distance data, then publishes corrected follower velocities.

The experiments were run on a laptop with an NVIDIA RTX 5070 Laptop GPU, 32 GB RAM, and an Intel Core Ultra 9 275HX CPU. Timing results should therefore be interpreted as implementation-level measurements on this hardware, not as hardware-independent solver benchmarks.

The main logged metrics are listed in Table 5.1. The safety margin used by the obstacle-distance node is 0.05 m. The penetration count $n_{d<0}$ counts samples with negative signed distance, not samples that are merely inside the 0.05 m safety margin.

5.2 Experiment 1: CBF On vs CBF Off

Experiment 1 compares the same table-approach motion with the CBF controller enabled and disabled. The cloth starts at $z = 0.50$ m and $y = -0.75$ m. The leaders move in the positive y direction at 0.05 m/s. The cloth grid is 56×56 cells, corresponding to 3249 particles.

Table 5.2 shows the main result. With CBF disabled, the cloth hits the table. With CBF enabled, there are no penetration samples. The followers lift from 0.50 m to about

Table 5.1: Primary metrics used in the experiment summaries.

Metric	Meaning
$\min d_{\min}$	Minimum signed cloth–obstacle distance. Negative values indicate penetration.
$n_{d<0}$	Number of samples with negative signed obstacle distance.
FPS	Mean simulation frame rate reported by Isaac Sim.
Step time	Mean simulation-frame processing time.
CBF time	Mean controller callback computation time.
Active steps	Number of logged steps where the obstacle CBF was active.
Minimum stretch h	Minimum held-point stretch barrier value. Negative values would indicate overstretching.
Follower height	Maximum follower-corner z position during the run.

Table 5.2: Experiment 1 CBF on/off results. The configured safety margin is 0.05 m.

Case	$\min d_{\min}$ (m)	$n_{d<0}$	FPS	Step (ms)	CBF (ms)	Active	Max follower z (m)
CBF on	0.0362	0	29.03	35.29	0.464	3141	0.654 / 0.656
CBF off	-0.0215	292	28.30	36.14	0.349	0	0.500 / 0.500

0.655 m, which is the main avoidance action. The minimum distance is still below the 0.05 m safety margin, so this result should be read as collision avoidance, not exact margin enforcement. Figure 5.1 plots the distance history, Figure 5.2 shows the corner trajectories, and Figures 5.3–5.4 show the visual CBF-off and CBF-on outcomes.

5.3 Experiment 2: Cloth-Resolution Sweep

Experiment 2 changes cloth resolution while keeping the physical cloth size fixed. Higher resolution gives smoother visual deformation, but it also increases simulation cost. The controller cost remains nearly constant because the current CBF-QP dimension does not grow with the cloth particle count.

The measured simulation rate drops from 40.42 Hz at 441 particles to 11.39 Hz at 16641 particles, as listed in Table 5.3. The minimum distance also decreases as the mesh becomes finer, because the cloth can sag. The 56×56 setting is a practical compromise for the current experiments: it gives visible sagging and contact behavior while staying close to 30 Hz on the tested hardware. Figure 5.5 shows the corresponding visual resolution sweep.

5.4 Experiment 3: Leader-Speed Sweep

Experiment 3 keeps the CBF enabled and varies leader speed. The tested speeds are 0.10, 0.40, and 1.00 m/s. The minimum logged distances are 0.0293 m, 0.0181 m, and 0.00115 m respectively. The first two cases avoid penetration but still do not maintain the configured

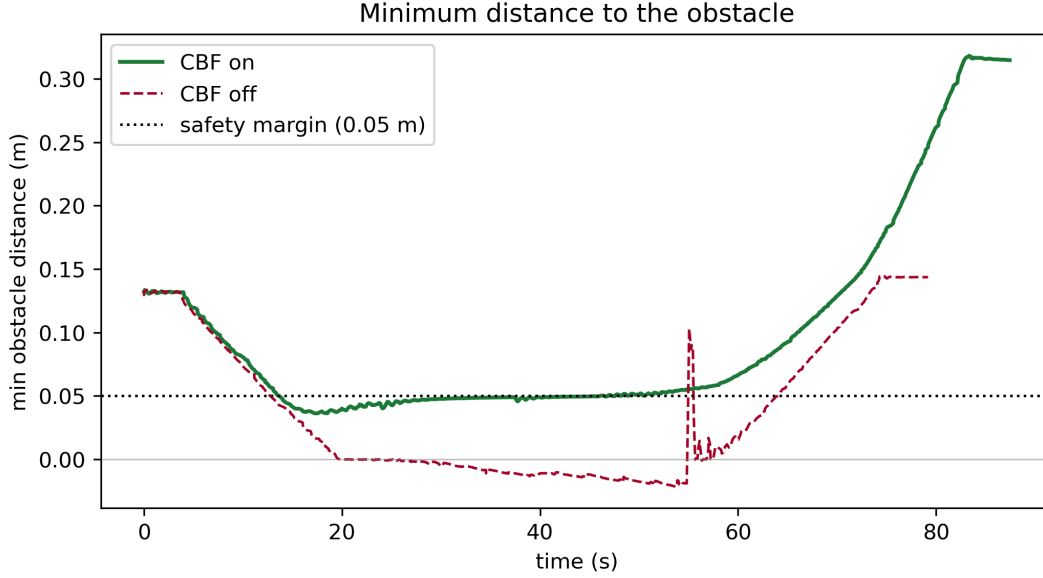


Figure 5.1: Experiment 1 minimum obstacle distance. CBF off leads to table penetration; CBF on avoids negative signed distance.

Table 5.3: Experiment 2 resolution-sweep results. The cloth physical size is held fixed while particle count increases.

Grid	Particles	FPS	Step (ms)	CBF (ms)	min $d_{\min}$ (m)	Active steps
20×20	441	40.42	24.85	0.457	0.1160	0
56×56	3249	29.36	34.88	0.452	0.0739	1922
72×72	5329	23.17	45.23	0.440	0.0550	2916
128×128	16641	11.39	99.37	0.427	0.0257	6697

0.05 m margin. The 1.00 m/s case should be treated as touching and as a failure to enforce a meaningful margin, even though the sampled signed-distance log did not become negative.

The failure mechanism is the interaction between fast leader motion, deformable-object swing, and the follower velocity bound. At 1.00 m/s, the leaders move much faster than the followers can respond under the 0.20 m/s command limit. The cloth also swings and locally contacts the obstacle before the geometric closest-point CBF correction reacts strongly enough. The chosen α gains only scale the correction from the currently observed barrier value, and the current geometric Jacobian does not predict the delayed cloth motion caused by the leader input. Therefore the 1.00 m/s run is useful as a stress test, not as a successful safety case.

The mean simulation rate remains near 29–30 Hz for these runs. The mean CBF solve time is about 0.46–0.48 ms. This again shows that the small external controller is not the runtime bottleneck. The clearance trend is plotted in Figure 5.6.

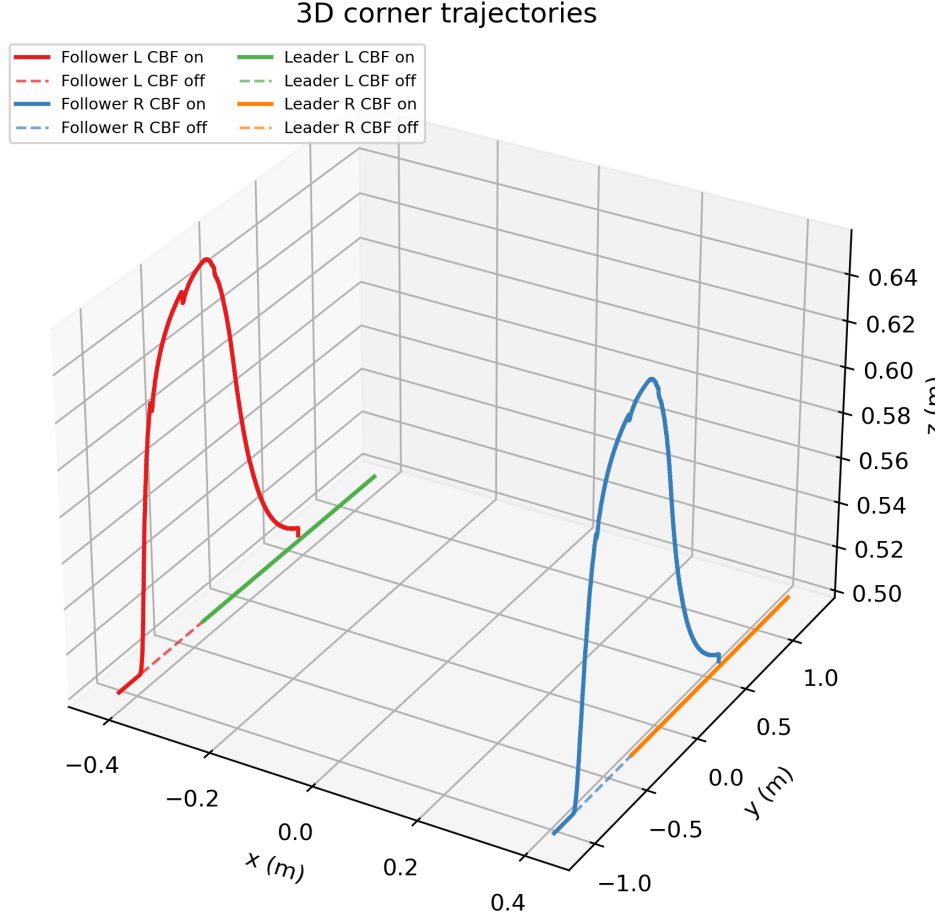


Figure 5.2: Experiment 1 corner trajectories. With CBF enabled, the follower corners move upward near the obstacle.

5.5 Experiment 4: Contact-Friction Sweep

Experiment 4 varies the Newton contact-friction parameter. The tested values are $\mu = 0.05$, 0.50, 1.00, and 4.00.

Low friction allows the fabric to slide more freely over the obstacle. Higher friction produces more local dragging, stronger wrinkling near contact, and more resistance to sliding. Figure 5.7 shows the visual comparison. These values are solver parameters, not calibrated real cloth-table friction coefficients.

5.6 Experiment 5: Material-Stiffness Sweep

Experiment 5 varies the Style3D stiffness preset. The tested presets are low, mid, and high stiffness cases. The mid preset is the default used in the main quantitative experiments, with in-plane stiffness values `tri_ke_weft`=30000, `tri_ke_warp`=30000, and `tri_ke_shear`=6000.

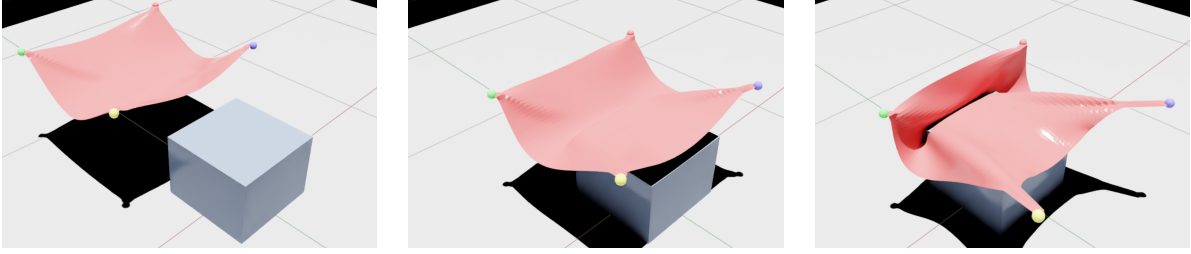


Figure 5.3: Experiment 1 CBF-off. The sheet is dragged into the table obstacle.

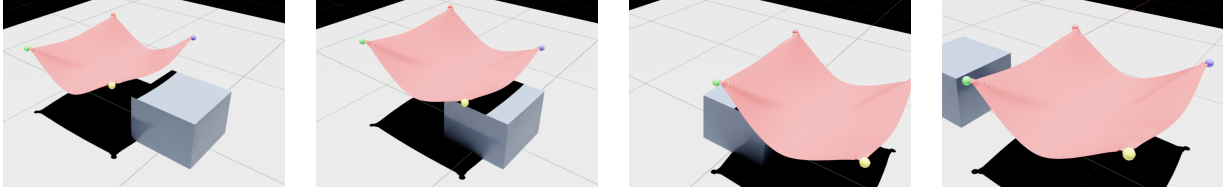


Figure 5.4: Experiment 1 CBF-on. The follower edge lifts as the table becomes active in the barrier constraint.

The low preset reduces stiffness by about one order of magnitude, and the high preset increases it by about one order of magnitude.

The low-stiffness sheet sags more and forms larger curvature. The mid-stiffness sheet gives the default compromise. The high-stiffness sheet behaves more like a rigid surface and deforms less under the same corner motion. Figure 5.8 shows the three stiffness cases. This confirms that the exposed Style3D stiffness parameters strongly affect the simulated cloth behavior.

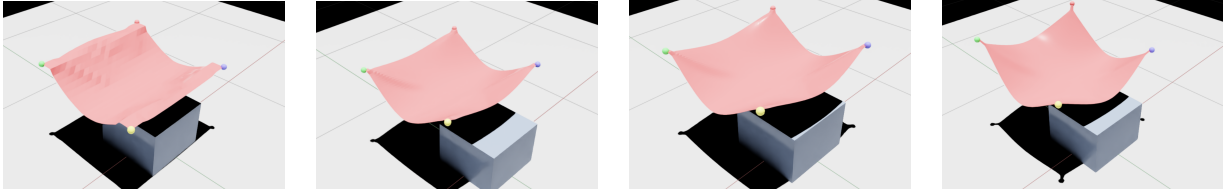


Figure 5.5: Experiment 2 screenshots for increasing cloth resolution: 20x20, 56x56, 72x72, 128x128.

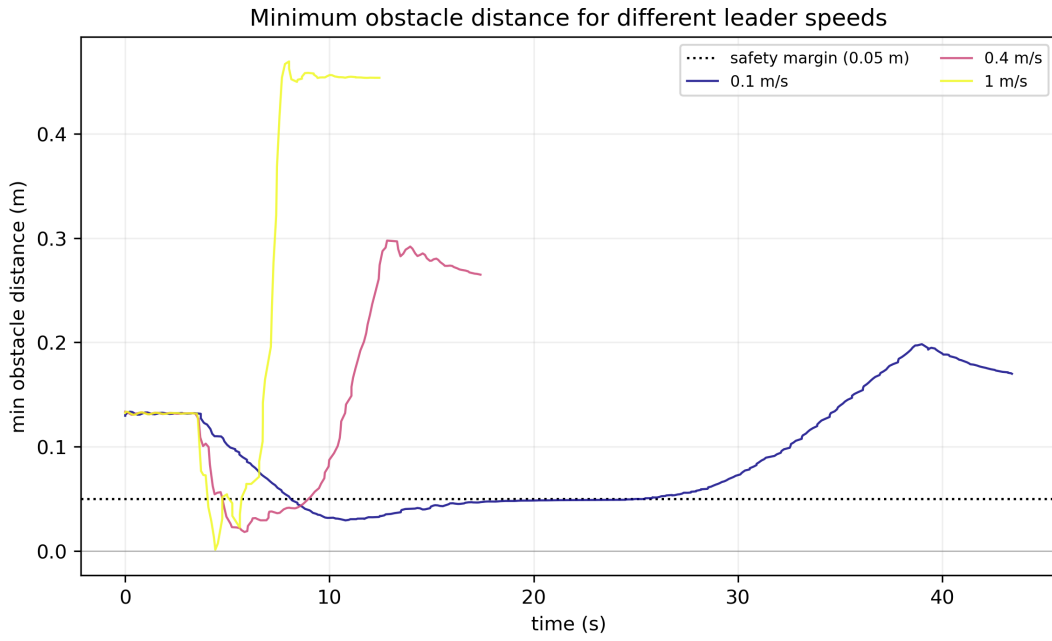


Figure 5.6: Experiment 3 minimum obstacle distance for three leader speeds. The 1.00 m/s run reaches only 1.15 mm clearance and is treated as touching/failure rather than successful margin enforcement.

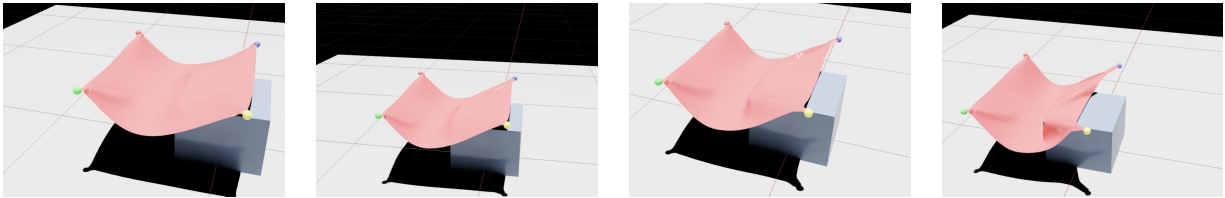


Figure 5.7: Experiment 4 contact-friction sweep. From left to right: $\mu = 0.05$, 0.50, 1.00, and 4.00.

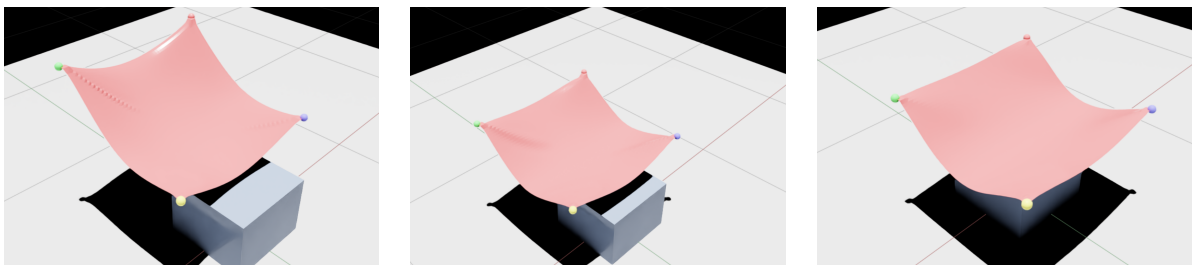


Figure 5.8: Experiment 5 stiffness sweep. From left to right: low, mid, and high stiffness presets.

Chapter 6

Discussion, Conclusion, and Future Work

6.1 Discussion

The experiments show that the Isaac Sim, Newton Style3D, ROS2, and external CBF controller pipeline is functional for simulated cloth manipulation. The main improvement over the predecessor ROS1/PBD setup is the visual and contact behavior of the simulated cloth. Newton Style3D produces smoother sagging, bending, and local contact deformation, so the simulated sheet looks closer to real fabric than the previous simplified model. However, this is still a qualitative improvement. No real fabric data was used to identify the parameters, so the report does not claim calibrated real-to-sim agreement.

The stiffness sweep shows that the Style3D parameters strongly affect the response. The mid preset, with `tri_ke_weft=30000`, `tri_ke_warp=30000`, and `tri_ke_shear=6000`, gave the most useful behavior for the tested scenes. Lower stiffness produced excessive sagging and larger curvature. Higher stiffness produced a sheet that behaved closer to a rigid surface. These ranges are physically meaningful only as solver parameters for the current simulation; they are not measured warp, weft, or shear constants for a specific real fabric.

The CBF controller prevented table penetration in the main CBF-on experiment, while the CBF-off baseline collided with the obstacle. This verifies the controller integration. At the same time, the experiments show that the controller enforces collision avoidance more reliably than it enforces the configured 0.05 m margin. The 1.00 m/s leader-speed case effectively failed: the logged minimum distance stayed slightly positive, but the 1.15 mm clearance is contact for practical purposes. The reason is not QP computation time. The limiting factors are the 0.20 m/s follower velocity bound, the chosen α gains, the geometric obstacle Jacobian, and the delayed swinging motion of the deformable cloth. In that case, the cloth can hit the obstacle before the CBF correction becomes strong enough.

For high resolutions, the runtime bottleneck is the cloth simulation, not the CBF-QP controller. Increasing cloth resolution improves visual smoothness but reduces simulation

rate substantially. The external controller computation remains below about 0.5 ms on average, while the simulation step time increases strongly with particle count.

6.2 Conclusion

This project developed a simulation-stage framework for obstacle-aware cloth manipulation. The work replaced the predecessor ROS1/PBD cloth model with an Isaac Sim and Newton Style3D pipeline, added anisotropic cloth parameters, connected the simulator to ROS2, and integrated an external CBF controller for follower-corner obstacle avoidance.

The first requirement in Section 1.3 was to simulate cloth-like behavior with anisotropic parameters. This requirement was met at the simulation level. The implemented Style3D backend exposes warp, weft, shear, bending, damping, friction, and contact settings, and the stiffness and friction sweeps show that these parameters visibly change the cloth response. The limitation is that the parameters are not yet calibrated against physical measurements.

The second requirement was to run fast enough for interactive or closed-loop use. This requirement was partially met. The 56×56 cloth setting runs near 30 Hz on the tested laptop and is usable for the reported closed-loop experiments. Higher resolution improves visual quality but drops the rate significantly, so the current implementation has a clear resolution-performance tradeoff.

The third requirement was to expose a clean ROS2 interface for commanding cloth corner points. This requirement was met. The simulator publishes corner poses and obstacle-distance information, receives leader and follower velocity commands, and supports the external controller through ROS2 topics. The package and topic structure can be reused for future real-robot integration.

The fourth requirement was to support CBF-style obstacle avoidance while leaving a path toward rollout-based Jacobian estimation and real-to-sim optimization. This requirement was partially met. The CBF controller successfully prevents penetration in the main on/off comparison, but it does not consistently preserve the intended safety margin under high leader speed or swinging cloth motion. The geometric Jacobian is therefore an interim solution, not the final safety mechanism.

Overall, the project establishes a stronger simulation and control foundation for cloth-manipulation research. The current system is not yet a real-fabric manipulation system. Its main value is that it moves the project from a simplified PBD sheet toward a more realistic, ROS2-compatible cloth simulation platform where future perception, parameter calibration, and robot deployment can be added.

6.3 Future Work

The next step is to close the perception-to-simulation loop. A depth-camera perception module should estimate the real cloth state and compare it with the simulated state. A

practical first version can use an RGB-D camera and an ArUco-tag layout to obtain sparse, metrically meaningful points on the cloth. The perception node would detect tags in the RGB image, sample registered depth near each tag, estimate local positions and normals, and optionally produce a coarse cloth point cloud or mask. The goal does not need to be full wrinkle-level reconstruction; it should be a compact state estimate sufficient for calibration.

The calibration objective can compare measured and simulated trajectories under the same corner motions, for example by minimizing

$$\min_{\boldsymbol{\theta}} \sum_t \|\mathbf{x}_t^{\text{real}} - \mathbf{x}_t^{\text{sim}}(\boldsymbol{\theta})\|^2, \quad (6.1)$$

where $\boldsymbol{\theta}$ contains Style3D stiffness, bending, damping, friction, mass, and contact parameters. In the near term, this can be done offline with finite-difference parameter sweeps or black-box optimization. Differentiable Style3D, if available for the needed scene, would make this loop more efficient.

The current geometric obstacle Jacobian should also be replaced or supplemented with auxiliary simulation rollouts. Instead of using only local geometry, the controller can perturb each follower command, simulate the short-horizon cloth response, and estimate a more accurate distance Jacobian from the resulting obstacle distances. This should improve cases where the cloth swings, folds, or contacts the obstacle before the present closest-point heuristic reacts.

The final target is real-robot deployment. The planned system will connect the simulation and controller to two robot manipulators, while a human acts as the leader. The OARbot model should also be added to the Isaac Sim environment so that the robots, cloth, obstacle, and controller can be tested in one integrated scene before physical experiments.

Bibliography

- [1] H. Yin, A. Varava, and D. Kragic, “Modeling, learning, perception, and control methods for deformable object manipulation,” *Science Robotics*, vol. 6, no. 54, p. eabd8803, 2021.
- [2] J. Zhu *et al.*, “Challenges and outlook in robotic manipulation of deformable objects,” *IEEE Robotics & Automation Magazine*, vol. 29, no. 3, pp. 67–77, 2022.
- [3] B. Aksoy and J. T. Wen, “Collaborative manipulation of deformable objects with predictive obstacle avoidance,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, Yokohama, Japan, 2024, pp. 16 766–16 772.
- [4] B. Aksoy, “A unified architecture and control framework for safe and collaborative human–robot manipulation of deformable objects,” Ph.D. dissertation, Rensselaer Polytechnic Institute, Troy, NY, USA, 2025.
- [5] Newton Contributors, “Newton physics documentation,” Online documentation, accessed May 2026. [Online]. Available: <https://newton-physics.github.io/newton/>
- [6] —, “SolverStyle3D and style3d cloth api reference,” Online documentation, accessed May 2026. [Online]. Available: https://newton-physics.github.io/newton/latest/api/_generated/newton.solvers.SolverStyle3D.html
- [7] D. Baraff and A. Witkin, “Large steps in cloth simulation,” in *Proceedings of SIGGRAPH 1998*, 1998, pp. 43–54.
- [8] T. Liu, A. W. Bargteil, J. F. O’Brien, and L. Kavan, “Fast simulation of mass-spring systems,” *ACM Transactions on Graphics*, vol. 32, no. 6, pp. 214:1–214:7, 2013.
- [9] K.-J. Bathe, *Finite Element Procedures*. Englewood Cliffs, NJ, USA: Prentice Hall, 1996.
- [10] M. Müller, B. Heidelberger, M. Hennix, and J. Ratcliff, “Position based dynamics,” in *Proceedings of the 3rd Workshop in Virtual Reality Interactions and Physical Simulation*, 2006, pp. 71–80.
- [11] M. Macklin, M. Müller, and N. Chentanez, “XPBD: Position-based simulation of compliant constrained dynamics,” in *Proceedings of Motion in Games*, 2016, pp. 49–54.

- [12] S. Bouaziz, S. Martin, T. Liu, L. Kavan, and M. Pauly, “Projective dynamics: Fusing constraint projections for fast simulation,” *ACM Transactions on Graphics*, vol. 33, no. 4, pp. 154:1–154:11, 2014.
- [13] NVIDIA Corporation, “Isaac sim newton physics backend documentation,” Online documentation, accessed May 2026. [Online]. Available: <https://docs.isaacsim.omniverse.nvidia.com/6.0.0/>
- [14] B. Aksoy, “deformable_manipulations_composite_layup: Ros1 package for composite layup deformable manipulation,” GitHub repository, accessed May 2026. [Online]. Available: https://github.com/burakaksoy/deformable_manipulations_composite_layup
- [15] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, “Control barrier function based quadratic programs for safety critical systems,” *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017.